

BIG DATA — COMPLETE REVISION SHEET

C-CAT Section B | Topics 1–10 | Phase 1

All key facts, comparisons & MCQ-ready points in one place

TOPIC 1 | BIG DATA CONCEPTS — THE 5 VS

■ Definition

- Big Data = data too large/fast/complex for traditional tools (Excel, normal DBs)
- Coined by Doug Laney (3 Vs: Volume, Velocity, Variety) in 2001; IBM added Veracity & Value

■ The 5 Vs — Quick Reference

V	One Word	Question It Answers	Key Exam Example
Volume	Size	How much data?	Facebook 100 PB photos; Google 20 PB/day
Velocity	Speed	How fast does data arrive?	6M Google searches/min; 500K tweets/min
Variety	Type	What formats?	Structured, Semi-structured, Unstructured
Veracity	Trust	How accurate/reliable?	Fake reviews, spam, typos, multi-language
Value	Usefulness	What insight can we get?	Hospital finding better drug combo

■ **Volume ≠ Value** — Volume = **SIZE** of data; Value = **USEFULNESS** extracted from it

■ **6th V = Variability** (meaning of data keeps changing) — less common but recognise it

■ Data at Rest = Volume | Data in Motion = Velocity

TOPIC 2 | TYPES OF DATA — STRUCTURED, SEMI-STRUCTURED, UNSTRUCTURED

Type	Format	Key Examples	Storage	% of World Data
Structured	Fixed rows & columns	SQL tables, Excel sheets	RDBMS (MySQL, Oracle)	~20%
Semi-Structured	Tags / key-value pairs	JSON, XML, Emails, HTML	NoSQL DBs, files	Small portion
Unstructured	No fixed format	Images, Videos, Audio, Social posts	Data Lakes, special storage	~80–90%

■ ~80–90% of world's data is **UNSTRUCTURED** — direct exam fact

■ **Excel file = STRUCTURED** (rows & columns). **Email = SEMI-STRUCTURED** (headers + free body)

- JSON & XML are the two most tested semi-structured formats
- Metadata = data about data (photo's GPS, date, camera model) — helps organise unstructured files
- Data Lake stores ALL types raw. Data Warehouse stores only structured, clean data

TOPIC 3 | BIG DATA PROCESSING — BATCH VS STREAMING

Feature	Batch Processing	Streaming Processing
When processed?	After collecting over time	Instantly as it arrives
Latency	HIGH — hours/days delay	LOW — milliseconds to seconds
Data volume	Large historical datasets	Continuous live data
Cost & Complexity	Lower cost, simpler	Higher cost, more complex
Best for	Monthly reports, billing, payroll	Fraud detection, live tracking, stock market
Main Tool	Apache Hadoop (MapReduce)	Apache Kafka, Apache Flink, Spark Streaming
Example	Bank monthly statement	UPI fraud detection

■ **Latency = time gap between data arriving and being processed**

■ **Apache Spark does BOTH batch AND streaming (micro-batch)** — commonly tested!

- Micro-batch = Spark collects data every few seconds → processes tiny batch → repeats
- Lambda Architecture = Batch layer + Speed (streaming) layer + Serving layer combined
- Streaming ≠ YouTube streaming. In Big Data, streaming = processing data in real-time as it flows
- Apache Kafka = message queue / pipeline for streaming data (delivers, doesn't process)

TOPIC 4 | NOSQL, BASE PROPERTIES & CAP THEOREM

■ NoSQL — 4 Types

Type	How Data Stored	Examples	Use Case
Key-Value Store	Unique key → value pairs	Redis, DynamoDB	Shopping cart, user sessions, caching
Document Store	JSON/XML documents (flexible cols)	MongoDB, CouchDB	User profiles, e-commerce, blog posts
Column-Family Store	Columns grouped in families	Cassandra, HBase	Time-series data, large-scale analytics
Graph Database	Nodes (entities) + Edges (relations)	Neo4j, Neptune	Social networks, fraud detection

■ BASE Properties (NoSQL)

- B = Basically Available — system always responds even if data slightly outdated
- A = Soft State — data may change during sync across machines (temporary in-between state)
- E = Eventually Consistent — all copies will EVENTUALLY match; not guaranteed instantly
- **BASE data is NOT wrong — it is temporarily out of sync but moves toward correctness**

■ CAP Theorem — Eric Brewer, 2000

- C = Consistency — all nodes show same data at same time
- A = Availability — system always responds to every request
- P = Partition Tolerance — system works even if network between machines breaks
- **Only 2 of 3 can be fully guaranteed. P is always required → real choice = CP or AP**

Database	CAP Choice	Meaning
MongoDB	CP	Prefers consistency over availability during network failure
Cassandra	AP	Always responds — may return slightly outdated data
HBase	CP	Prioritises consistency
CouchDB	AP	Prioritises availability
Redis	CP	Prioritises consistency

■ **CAP = C, A, P — NOT C, I, P. A = Availability (not I)**

■ **NoSQL = Not Only SQL. Schema-less = flexible, add fields anytime. Horizontal scaling = add more machines**

TOPIC 5 | RDBMS, ACID PROPERTIES & BASIC SQL

■ RDBMS Key Terms

Term	Meaning	Example
Table / Relation	Grid of rows and columns for one entity	Student table
Row / Tuple	One single record	Details of one student
Column / Attribute	One property being tracked	Name, Age, Roll No
Primary Key (PK)	Unique identifier — no NULL, no duplicate	Roll No in Student table
Foreign Key (FK)	Column linking to PK of another table	Course_ID in Student table
Schema	Blueprint of table — defined before data entry	Columns + data types + rules

■ ACID Properties

Property	One Word	Meaning	Violated → What Happens
Atomicity	All-or-Nothing	Every step completes or entire txn rolled back	Half transfer — money disappears
Consistency	Valid State	DB moves from one valid state to another valid state	Invalid data enters DB
Isolation	Independence	Concurrent transactions don't interfere	Two users withdraw same money
Durability	Permanent	Committed data survives crashes	Confirmed txn lost after crash

■ SQL Command Categories

Category	Full Form	Commands	What They Do
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Define/modify table structure
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Work with data inside tables
DCL	Data Control Language	GRANT, REVOKE	Control user permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

■ **DROP = deletes table + structure. TRUNCATE = deletes all rows but keeps structure**

■ **DELETE can use WHERE (specific rows). TRUNCATE removes ALL rows, no conditions**

- Primary Key cannot be NULL — direct exam fact
- Normalisation = organising DB to reduce redundancy. Normal Forms: 1NF, 2NF, 3NF

TOPIC 6 | DATA WAREHOUSE, OLAP & OLTP

■ OLTP vs OLAP

Feature	OLTP	OLAP
Full Form	Online Transaction Processing	Online Analytical Processing
Purpose	Day-to-day operations	Analysis & decision-making
Operations	INSERT, UPDATE, DELETE	SELECT (complex queries)
Rows per query	Few rows (1–10)	Millions of rows
Response time	Milliseconds	Seconds to minutes
Data type	Current, real-time data	Historical data
DB Design	Normalised	Denormalised
Users	Clerks, cashiers, customers	Analysts, managers, executives
Backed by	RDBMS (MySQL, Oracle)	Data Warehouse (Redshift, BigQuery)
Examples	ATM, ticket booking, UPI	Sales trend reports, forecasting

■ Data Warehouse

- Centralised historical store — collects from multiple OLTP sources via ETL
- Stores only STRUCTURED, CLEAN data. Data Lake stores all types raw
- Popular tools: Amazon Redshift, Google BigQuery, Snowflake, Teradata
- Data Mart = smaller, department-focused version of Data Warehouse (e.g. Sales Mart, HR Mart)

■ OLAP Operations — All 5 Are Exam-Worthy

Operation	Direction	What It Does	Example
Roll Up	Bottom → Top	Detail to summary	Monthly sales → Quarterly → Yearly
Drill Down	Top → Bottom	Summary to detail	Yearly → Quarterly → Monthly → Daily
Slice	One dimension	Fix one value, get 2D view	Only Year 2023 data
Dice	Multi-dim	Select sub-cube across dimensions	Smartphones in MH in Q1 2023
Pivot	Rotate	Swap rows and columns	Switch time and region axes

■ OLAP Types

- ROLAP = Relational OLAP — runs on relational DB using SQL
- MOLAP = Multidimensional OLAP — stores in special cube format, fastest
- HOLAP = Hybrid OLAP — combination of ROLAP + MOLAP
- Roll Up ≠ Drill Down: Roll Up goes UP to bigger summary; Drill Down goes DOWN to detail
- OLTP = Normalised design. OLAP/Warehouse = Denormalised design — frequently tested

■ Schema Types in Data Warehouse

Schema	Structure	Query Speed	Storage	Use When
Star Schema	1 Fact Table + flat Dimension Tables	Faster (fewer JOINS)	More (some redundancy)	Speed is priority
Snowflake Schema	Fact Table + normalised Dimension sub-tables	Slower (more JOINS)	Less (normalised)	Storage is priority
Galaxy Schema	Multiple Fact Tables sharing Dimensions	Complex	Enterprise scale	Large enterprise DW

- Fact Table = measurable data (amounts, quantities). Dimension Table = descriptive context (names, dates, locations)

TOPIC 7 | ETL VS ELT

Feature	ETL	ELT
Full Form	Extract → Transform → Load	Extract → Load → Transform
When Transform?	BEFORE loading — in Staging Area	AFTER loading — inside target system
Target System	Data Warehouse	Data Lake / Cloud Data Warehouse
Data Types Handled	Mostly structured data	Structured + Semi-structured + Unstructured
Raw Data Preserved?	NO — discarded after transform	YES — always available
Loading Speed	Slower — must transform first	Faster — load raw immediately
Best For	Traditional, structured, defined reports	Big Data, unstructured, flexible AI analysis
Era	Traditional / On-premise	Modern / Cloud-based

Feature	ETL	ELT
Example Tools	Informatica, Talend, IBM DataStage	Apache Spark, dbt, Google Dataflow

■ ETL Steps Detail

- EXTRACT — Pull from multiple sources (OLTP DBs, CSV, APIs, legacy systems)
- TRANSFORM (in Staging Area) — Clean, filter, convert types, aggregate, merge, apply business rules
- LOAD — Full Load (everything, first time) OR Incremental Load (only new/changed data)
- Staging Area = temporary workspace between source and DW. Deleted/archived after load

■ Key Terms

- GIGO = Garbage In, Garbage Out — dirty input → wrong analysis output
- Incremental Load = add only new/changed records since last run (more efficient)
- CDC (Change Data Capture) = tracks only changed rows for incremental load
- Data Pipeline = complete automated flow: collect → clean → transform → store → schedule
- ELT is NOT just ETL with steps reordered — it is a fundamentally different architecture for cloud/Big Data
- ELT requires structured data. ELT handles ALL types — don't swap these in the exam

TOPIC 8 | DATA CLEANSING & DATA MODELING

■ Data Cleansing — Problems & Fixes

Problem	What It Means	Fix
Missing Values	Fields with no data (NULL)	Delete row / Imputation / Flag as Unknown
Duplicate Records	Same entity entered multiple times	Keep one, delete rest (merge if needed)
Inconsistent Format	Same data in different formats	Standardise to one format (dates, names)
Invalid Data	Impossible values (age=450, rating=7.5/5)	Correct, delete, or flag for review
Outliers	Values abnormally far from rest	Remove if error; keep if genuine extreme
Inconsistent Values	Same concept stored differently (M/Male/male)	Standardise to one canonical value
Misspellings	Typos in text fields (Mumbay)	Match against standard list, correct
Irrelevant Data	Data not needed for analysis	Filter/remove from dataset

- Imputation = fill missing values with mean / median / mode of that column
- Data Profiling = Step 1 — examine dataset to discover quality problems before fixing
- Data Validation = final check after cleaning — confirm data meets quality standards
- Data Wrangling / Munging = broader term including cleansing + reshaping + enriching
- Data Cleansing ≠ Data Transformation. Cleansing = fix quality issues. Transform = restructure for purpose

■ Three Levels of Data Modeling

Level	Name	Detail Level	Shows	Audience
1	Conceptual	Abstract / Big picture	Entities + relationships only (no attributes)	Business managers, stakeholders
2	Logical	Medium detail	Attributes + relationships, technology-independent	Data architects, analysts
3	Physical	Most detailed	Exact columns, data types, constraints, indexes for specific DB	Developers, DBAs

- ERD (Entity Relationship Diagram) = visual diagram of Data Model — entities as rectangles, relationships as lines
- Entity = real-world object (Student, Course). Attribute = property (Name, Age). Relationship = connection between entities
- Data Integrity = data remains accurate, consistent, trustworthy throughout lifecycle

TOPIC 9 | BIG DATA FRAMEWORKS — HADOOP, HIVE & SPARK

■ Apache Hadoop

- Created by Doug Cutting & Mike Cafarella in 2006. Inspired by Google's GFS + MapReduce papers
- Two core components: HDFS (storage) + MapReduce (processing)

HDFS Component	Role	Key Facts
NameNode	Master — stores metadata only (where blocks live)	Single point of coordination; does NOT store actual data
DataNode	Worker — stores actual data blocks	Multiple DataNodes across cluster
Block	Fixed chunk of a file	Default size = 128 MB (older = 64 MB)
Replication	Copy of each block on multiple machines	Default replication factor = 3 (fault tolerance)

- HDFS = Write Once Read Many — not designed for frequent updates
- MapReduce phases: MAP (split & tag key-value pairs) → SHUFFLE/SORT → REDUCE (aggregate)
- MapReduce is SLOW — reads/writes to disk between every step

- YARN = Yet Another Resource Negotiator — manages cluster resources (added in Hadoop 2.0)

■ Hadoop Ecosystem — Key Tools

Tool	Purpose
HDFS	Distributed file storage
MapReduce	Batch processing engine (slow, disk-based)
YARN	Resource manager — allocates CPU & memory to jobs
Hive	SQL-like querying on Hadoop (HiveQL)
HBase	NoSQL database on top of HDFS
Sqoop	Transfers data between HDFS and RDBMS
Flume	Collects streaming log data into HDFS
Pig	Scripting language (Pig Latin) for data transformation on Hadoop
ZooKeeper	Coordination service for distributed systems

■ Apache Hive

- Created by Facebook in 2008. Sits on top of Hadoop
- Uses HiveQL — SQL-like language. Converts queries to MapReduce jobs automatically
- Schema on Read (not Schema on Write) — structure applied when reading, not storing
- NOT for real-time — batch analysis only. Data stored in HDFS

■ **Hive is NOT a database — it is a query layer on top of HDFS**

■ Apache Spark

- Created at UC Berkeley AMPLab in 2009. Open-sourced 2010. Apache project 2013
- Up to 100x faster than MapReduce because it processes data IN-MEMORY (RAM)
- RDD = Resilient Distributed Dataset — core data structure (Resilient + Distributed + Dataset)

Spark Component	Purpose
Spark Core	Foundation — memory management, fault recovery, task scheduling
Spark SQL	SQL queries on structured data; uses DataFrame API; reads Hive tables
Spark Streaming	Real-time processing via MICRO-BATCHING (not true real-time)
MLlib	Built-in machine learning library for distributed model training
GraphX	Graph-structured data processing (social networks)

■ **Spark does NOT replace Hadoop — it complements it (uses HDFS + YARN, replaces MapReduce)**

■ **Spark Streaming = MICRO-BATCH (few seconds). Apache Flink = TRUE real-time**

- PySpark = Python API for Spark. DataFrame = distributed table-like structure
- Kafka + Spark Streaming often used together: Kafka delivers stream, Spark processes it

Feature	Hadoop MapReduce	Spark	Hive
Processing	Disk-based (slow)	In-memory (fast)	Converts to MapReduce/Spark
Speed	Slow	Up to 100x faster	Slow
Streaming	No	Yes (micro-batch)	No
ML Support	No	Yes (MLlib)	No
Created	2006	2009	2008
Created by	Doug Cutting	UC Berkeley AMPLab	Facebook
Language	Java	Scala, Python, Java, R	HiveQL (SQL-like)

TOPIC 10 | BIG DATA PROGRAMMING LANGUAGES — PYTHON, JAVA & SCALA

Feature	Python	Java	Scala
Created by	Guido van Rossum	James Gosling (Sun)	Martin Odersky (EPFL)
Year	1991	1995	2004
Name origin	Monty Python (comedy)	Java coffee (cup logo)	Scalable Language
Runs on	Python interpreter	JVM	JVM
Type system	Dynamically typed	Statically typed	Statically typed
Execution	Interpreted (slower)	Compiled to bytecode (fast)	Compiled to bytecode (fast)
Ease of learning	Very easy	Moderate	Difficult (steep curve)
Big Data role	Analytics, ML, PySpark	Hadoop MapReduce, enterprise	Native Spark, streaming

Feature	Python	Java	Scala
Key framework	PySpark, TensorFlow, Pandas	Hadoop, Kafka	Apache Spark
Code verbosity	Very concise (1 line)	Very verbose	Concise
Functional prog.	Partial support	Limited	Full support

■ Key Language Facts

- Hadoop is written in JAVA → Java = native language of Hadoop ecosystem
- Spark is written in SCALA → Scala gives best Spark performance (no translation layer)
- PySpark = Python API for Spark → reason Python dominates Big Data analytics despite being slower
- Java + Scala both run on JVM. Python does NOT run on JVM
- Dynamically typed (Python) = no need to declare variable types. Statically typed (Java/Scala) = must declare types
- Functional programming (Scala) = immutable data + functions as first-class citizens — safer for distributed systems
- R language = statistical computing, used by researchers. Spark supports R via SparkR

■ Python's Key Big Data Libraries

Library	Purpose
NumPy	Fast numerical computing on large arrays
Pandas	Data manipulation & analysis (like Excel in Python)
Matplotlib	Data visualisation — charts and graphs
Scikit-learn	Machine learning algorithms
TensorFlow	Deep learning and neural networks (Google)
PyTorch	Deep learning (Facebook/Meta)
PySpark	Python API for Apache Spark

- Python is slower than Java/Scala but PySpark offloads heavy processing to Spark's Scala engine
- Scala ≠ scripting language — it is a full compiled language running on JVM